

Day 8 - Facilitation Guide

(Dynamic HTML with JavaScript)

INDEX

- I. Recap of the previous session**
- II. Document Object Model (DOM)**
- III. Handling form submissions**
- IV. Navigation Control**
- V. Event Driven Programming Model in JavaScript**

(2 hrs) ILT

I. Recap of the previous session

In the last session we taught you many important concepts of JavaScript. Here's a brief introductory recap session covering the key concepts you've taught in the last session:

Control flow statements

Conditional statements allow you to make decisions in your code based on whether a condition is true or false. The most common structures are if, else if, and else. They help control the flow of your program.

Loops

Loops are used to repeatedly execute a block of code. for loops are ideal when you know the number of iterations, while while loops are suitable when the number of iterations is unknown.

Functions:

Functions are reusable blocks of code that perform a specific task. They help in modularizing code and promoting code reusability.

Arrays and objects are data structures. Arrays store ordered collections of values, and objects store key-value pairs.

Objects consist of key-value pairs, where keys are strings or symbols. Properties can be accessed using dot notation or bracket notation.

JSON Notation and Parsing:

JSON (JavaScript Object Notation) is a lightweight data interchange format. It is easy for humans to read and write and easy for machines to parse and generate.

In this chapter we will continue our discussion on JavaScript and explain one of the very important concepts - Document Object Model or DOM.

II. Document Object Model (DOM)

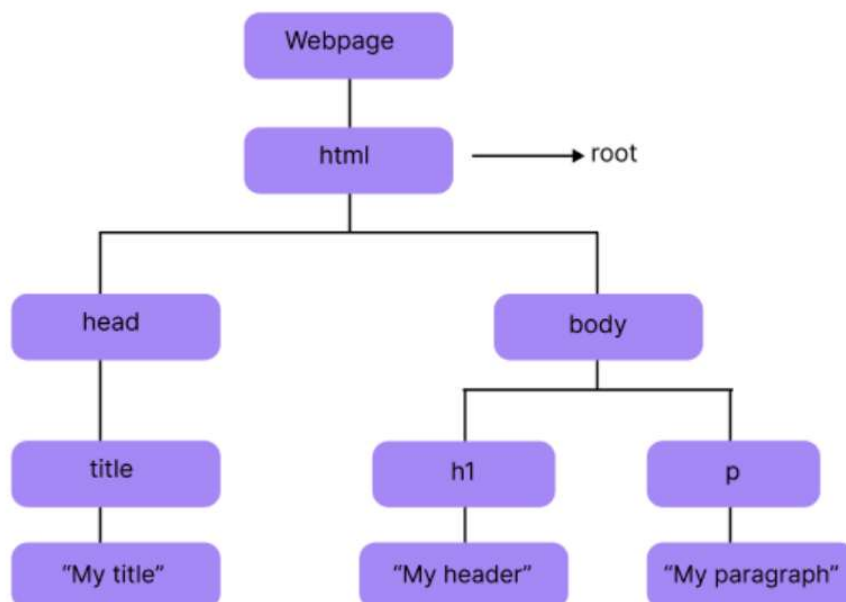
The Document Object Model (DOM) is a programming interface for web documents. It represents the structure of a document as a tree of objects, where each object corresponds to a part of the document, such as elements, attributes, and text content. JavaScript interacts with the DOM to dynamically update and modify web pages.

Understanding the DOM

To understand DOM we will first explain the key concepts in DOM:

Document Tree:

The DOM represents an HTML or XML document as a hierarchical tree structure. Each node in the tree corresponds to an element, attribute, or piece of text in the document.



Nodes:

Nodes are the fundamental building blocks of the DOM tree. Common types of nodes include elements, attributes, text, and comments.

Elements:

Elements are the HTML or XML tags in the document, such as <div>, <p>, or <a>. Elements are represented as element nodes in the DOM tree.

Attributes:

Attributes are additional information within HTML or XML tags.
They are represented as attribute nodes in the DOM tree.

Text Nodes:

Text nodes represent the text content of elements.

For example, the text between `<p>` and `</p>` tags is represented as a text node in the DOM.

Manipulation:

The DOM provides methods and properties for dynamically manipulating the content, structure, and style of a document.

Developers can add, modify, or remove elements and attributes, change styles, and update text content.

Traversal:

Developers can traverse the DOM tree to navigate between nodes.

Common traversal methods include accessing parent, child, and sibling nodes.

Example:

Let's create a simple ToDo List using DOM :

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <style>
    body {
      font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
      margin: 0;
      padding: 0;
      box-sizing: border-box;
      background-color: #e0f2f1;
      color: #333;
    }

    .container {
      max-width: 600px;
      margin: 50px auto;
      padding: 20px;
      background-color: #fff;
      box-shadow: 0 0 10px rgba(0, 0, 0, 0.1);
      border-radius: 8px;
    }

    h1 {
      text-align: center;
```

```

    color: #4CAF50;
}

.controls {
    margin-top: 20px;
    display: flex;
    justify-content: space-between;
}

input[type="text"] {
    padding: 10px;
    flex-grow: 1;
    border: 1px solid #ddd;
    border-radius: 4px;
}

button {
    padding: 10px 20px;
    cursor: pointer;
    background-color: #4CAF50;
    color: #fff;
    border: none;
    border-radius: 4px;
}

.task {
    background-color: #dff0d8;
    margin-bottom: 10px;
    padding: 15px;
    display: flex;
    justify-content: space-between;
    align-items: center;
    border: 1px solid #4CAF50;
    border-radius: 4px;
}

.task button {
    background-color: #d9534f;
    color: #fff;
    border: none;
    padding: 8px 12px;
    cursor: pointer;
    border-radius: 4px;
}
</style>
<title>To-Do List</title>

```

```

</head>
<body>
  <div class="container">
    <h1>To-Do List</h1>
    <div id="tasks-container">
      <!-- Tasks will be added dynamically here -->
    </div>
    <div class="controls">
      <input type="text" id="taskInput" placeholder="Add a new task">
      <button id="addTask">Add Task</button>
    </div>
  </div>
  <script>
    document.addEventListener('DOMContentLoaded', () => {
      const tasksContainer = document.getElementById('tasks-container');
      const taskInput = document.getElementById('taskInput');
      const addTaskButton = document.getElementById('addTask');

      addTaskButton.addEventListener('click', () => {
        const taskText = taskInput.value.trim();

        if (taskText !== "") {
          addTask(taskText);
          taskInput.value = "";
        }
      });

      function addTask(text) {
        const taskDiv = document.createElement('div');
        taskDiv.classList.add('task');

        const taskText = document.createElement('span');
        taskText.textContent = text;

        const deleteButton = document.createElement('button');
        deleteButton.textContent = 'Delete';
        deleteButton.addEventListener('click', () => {
          taskDiv.remove();
        });

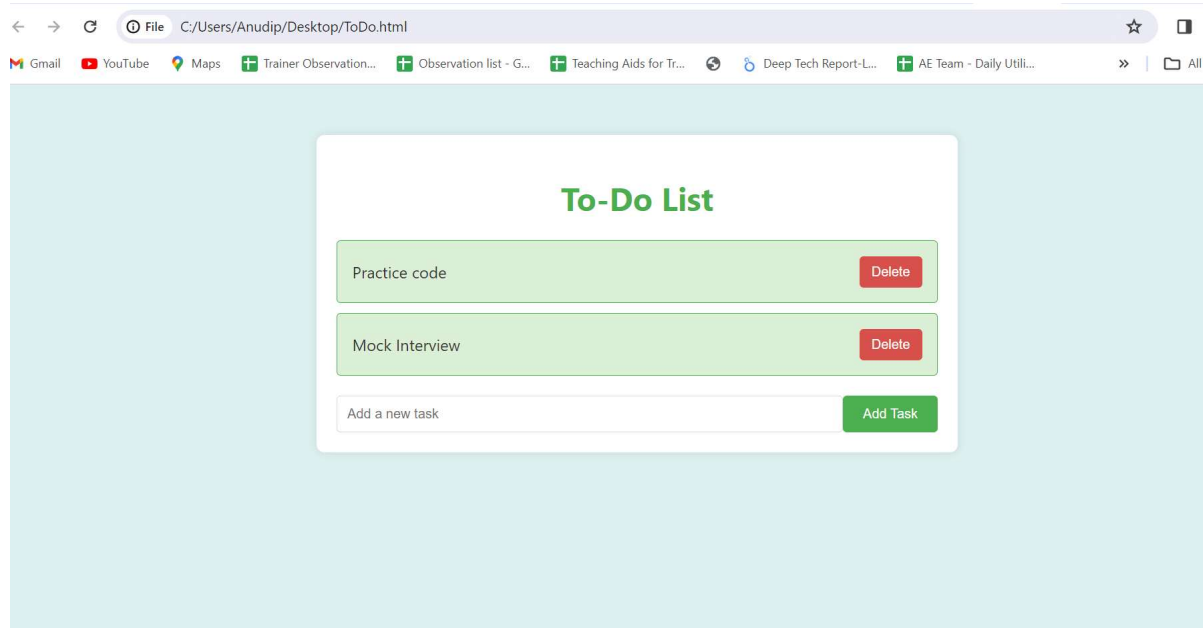
        taskDiv.appendChild(taskText);
        taskDiv.appendChild(deleteButton);

        tasksContainer.appendChild(taskDiv);
      }
    });
  </script>

```

```
</script>
</body>
</html>
```

Output:



In this example:

- The HTML file includes a button element with the ID "Add Task".
- The JavaScript code uses `document.getElementById` to obtain a reference to the button.
- `DOMContentLoaded` Event Listener ensures that the JavaScript code runs after the HTML document has been fully loaded.
- `addTask` Function dynamically creates a new task div, task text element, and delete button, Appends these elements to the tasks container.
- `Delete Button Functionality` removes the corresponding task div when clicked.

III. Handling Form Submission

Form submission refers to the process of sending the data entered by the user in the form to a server for further processing. When a user clicks the submit button in an HTML form, the browser collects the values from the form fields and sends them to the server specified in the action attribute of the `<form>` element using the HTTP method specified in the method attribute.

There are two common HTTP methods used for form submission:

GET: Appends the form data to the URL as a query string. It's suitable for small amounts of data, but not recommended for sensitive information or large data sets.

```
<form action="/submit_form" method="get">
  <!-- Form elements go here -->
  <input type="submit" value="Submit">
</form>
```

POST: Sends the form data as part of the HTTP request body. It's more secure and suitable for larger amounts of data or sensitive information like passwords.

```
<form action="/submit_form" method="post">
  <!-- Form elements go here -->
  <input type="submit" value="Submit">
</form>
```

<form>: The container for all the form elements. It includes attributes like action (the URL where the form data will be submitted) and method (the HTTP method used for the form submission, commonly "GET" or "POST").

Simple get and post request using fetch API:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <meta http-equiv="X-UA-Compatible" content="ie=edge">
  <title>Get and Post request</title>
  <style>
    body {
      font-family: Arial, sans-serif;
      margin: 0;
      padding: 0;
      background-color: #f4f4f4;
      text-align: center;
      padding: 20px;
    }

    h1 {
      color: #333;
    }

    pre {
      text-align: left;
      background-color: #fff;
      padding: 10px;
      border-radius: 8px;
      box-shadow: 0 0 10px rgba(0, 0, 0, 0.1);
      margin-top: 20px;
```

```
        max-width: 600px;
        margin-left: auto;
        margin-right: auto;
    }
</style>
</head>
```

```
<body>
```

```
    <h1>Simple Get and POST request using fetch API method by making a custom
    library</h1>
```

```
    <pre id="result"></pre>
```

```
    <script>
        class EasyHTTP {

            // Make an HTTP GET Request
            async get(url) {

                // Awaiting for fetch response
                const response = await fetch(url);

                // Awaiting for response.json()
                const resData = await response.json();

                // Returning result data
                return resData;
            }

            // Make an HTTP POST Request
            async post(url, data) {

                // Awaiting for fetch response and
                // defining method, headers and body
                const response = await fetch(url, {
                    method: 'POST',
                    headers: {
                        'Content-type': 'application/json'
                    },
                    body: JSON.stringify(data)
                });

                // Awaiting response.json()
                const resData = await response.json();
```



```

        // Returning result data
        return resData;
    }
}

// Instantiating EasyHTTP
const http = new EasyHTTP;

// Get prototype method
http.get('https://jsonplaceholder.typicode.com/users')

    // Resolving promise for response data
    .then(data => displayResult('GET Request:', data))

    // Resolving promise for error
    .catch(err => console.error('GET Error:', err));

// Data for post request
const postData = {
    name: 'selmon_bhoi',
    username: '_selmon',
    email: 'selmonbhoi@gmail.com'
};

// Post prototype method
http.post(
    'https://jsonplaceholder.typicode.com/users',
    postData)

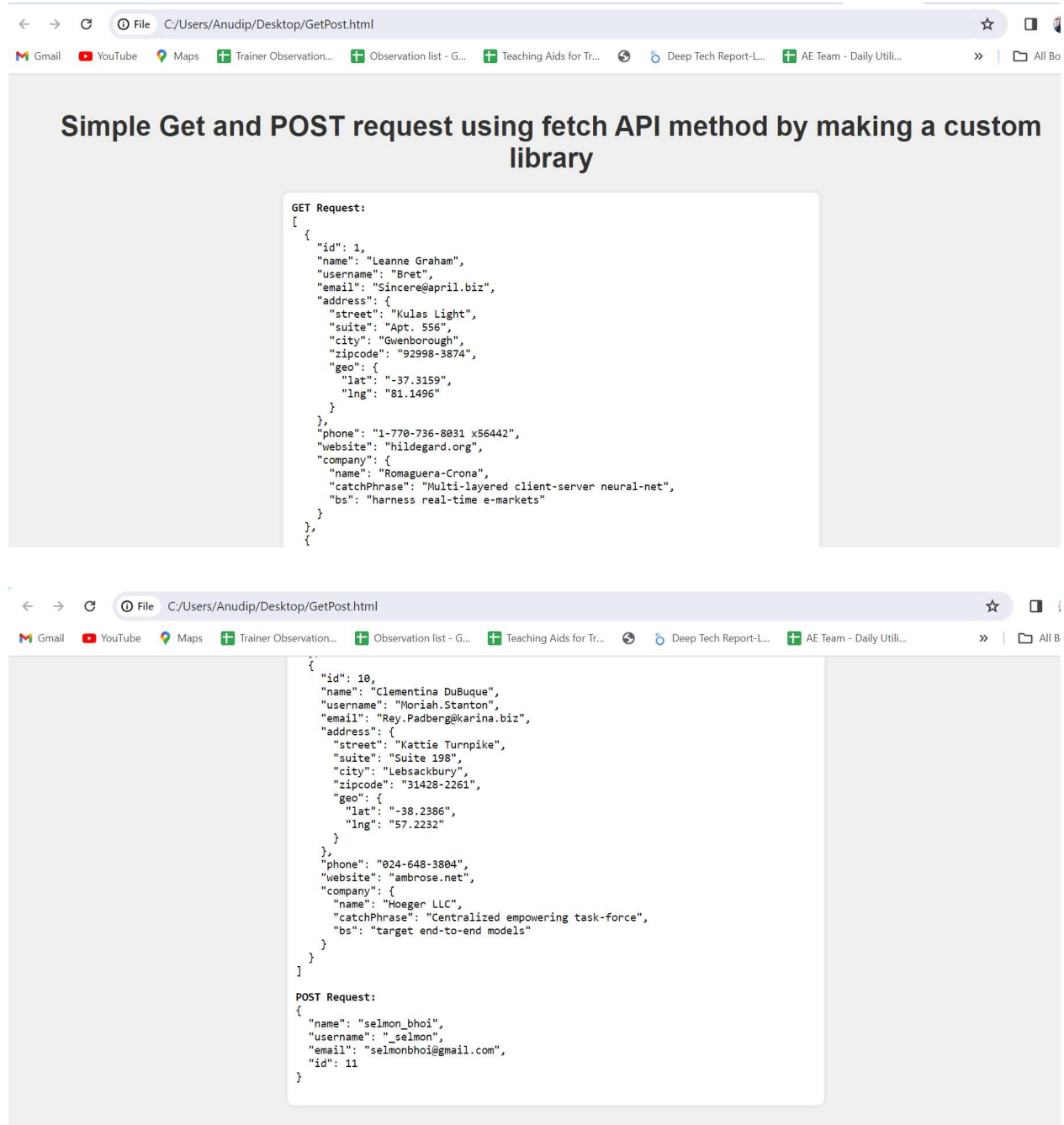
    // resolving promise for response data
    .then(data => displayResult('POST Request:', data))

    // resolving promise for error
    .catch(err => console.error('POST Error:', err));

// Function to display results
function displayResult(title, data) {
    const resultDiv = document.getElementById('result');
    resultDiv.innerHTML += `<strong>${title}</strong><br>${JSON.stringify(data,
null, 2)}<br><br>`;
}
</script>
</body>
</html>

```

Output:



IV. Navigation control

Navigation control using JavaScript typically involves changing or controlling the navigation behavior of a web page. This can include tasks like redirecting to another page, changing the URL without reloading the page (using the History API), and responding to navigation events. Below are some common scenarios with examples:

1. Redirecting to Another Page:

Example: Basic Page Redirection

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Page Redirection</title>
</head>
<body>

  <h1>Welcome to the Home Page</h1>
  <button onclick="redirectToAboutPage()">Go to About Page</button>

  <script>
    function redirectToAboutPage() {
      // Redirect to another page
      window.location.href = "about.html";
    }
  </script>
</body>
</html>
```

Output:



2. Changing the URL Without Reloading the Page (Using History API):

Example: Changing URL Dynamically

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Changing URL Dynamically</title>
</head>
```

```

<body>

  <h1 id="pageTitle">Home Page</h1>
  <button onclick="changeURL()">Go to About Page</button>

  <script>
    function changeURL() {
      // Change the URL without reloading the page
      history.pushState({}, "About Page", "about.html");
      document.getElementById("pageTitle").textContent = "About Page";
    }
  </script>
</body>
</html>

```

Output:



Home Page

Go to About Page

Let's take an example of a book gallery and create a web page using container class.

3. Responding to Navigation Events:

Example: Handling Back and Forward Buttons

```

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Navigation Events</title>
</head>
<body>

  <h1>Navigation Events Example</h1>
  <button onclick="goBack()">Go Back</button>
  <button onclick="goForward()">Go Forward</button>

  <script>
    function goBack() {
      // Navigate back in history

```

```

        window.history.back();
    }

    function goForward() {
        // Navigate forward in history
        window.history.forward();
    }

    // Event listener for navigation changes
    window.addEventListener("popstate", function(event) {
        alert("Navigation state changed!");
    });
</script>

</body>
</html>

```

Output:



Let's take the previous example of a book gallery and add the navigation.

index.html

```

<!DOCTYPE html>
<html lang="en">

<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Online Book Store Navigation</title>
  <link rel="stylesheet"
href="https://cdn.jsdelivr.net/npm/bootstrap@5.2.3/dist/css/bootstrap.min.css">
  <style>
    .container a {
      color: #fff;
    }
  </style>
</head>
<body>
  <nav class="navbar navbar-expand-lg navbar-light bg-dark ">
    <div class="container">
      <a class="navbar-brand" href="nav.html">Book Gallery</a>

```

```

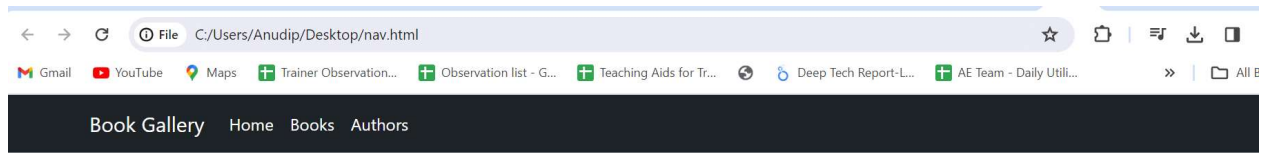
    <button class="navbar-toggler" type="button" data-toggle="collapse"
data-target="#navbarNav"
    aria-controls="navbarNav" aria-expanded="false" aria-label="Toggle navigation">
    <span class="navbar-toggler-icon"></span>
</button>
<div class="collapse navbar-collapse">
    <ul class="navbar-nav ml-auto">
        <li class="nav-item">
            <a class="nav-link" href="nav.html">Home</a>
        </li>
        <li class="nav-item">
            <a class="nav-link" href="Book_gallery.html">Books</a>
        </li>
        <li class="nav-item">
            <a class="nav-link">Authors</a>
        </li>
    </ul>
</div>
</div>
</nav>

<div class="container">
    <h3 class="text-center" style="padding-top: 20px; font-family: 'Arial', sans-serif; color:
#4CAF50;">About</h3>
    <p style="font-size: 18px; line-height: 1.6; text-align: justify; color: #333; font-family:
'Georgia', serif;">
        Embark on a literary odyssey through our online Book Gallery, where the written
word comes to life and stories unfold like vibrant canvases. Immerse yourself in a world
where every turn of the page opens a portal to new adventures, diverse perspectives,
and the boundless creativity of remarkable authors.
        <br><br>
        Our curated collection spans genres, inviting you to explore the realms of fiction,
non-fiction, fantasy, and beyond. Whether you're a seasoned reader or just beginning
your literary journey, the Book Gallery is a sanctuary for book lovers and a celebration
of the written artistry that enriches our lives.
        <br><br>
        Join us on this literary exploration, where every book is a treasure waiting to be
discovered, and every author is an architect of captivating narratives. The Book Gallery
awaits your presence—where stories breathe, ideas flourish, and the magic of words
unfolds.
    </p>
</div>

</body>

</html>

```



About

Embark on a literary odyssey through our online Book Gallery, where the written word comes to life and stories unfold like vibrant canvases. Immerse yourself in a world where every turn of the page opens a portal to new adventures, diverse perspectives, and the boundless creativity of remarkable authors.

Our curated collection spans genres, inviting you to explore the realms of fiction, non-fiction, fantasy, and beyond. Whether you're a seasoned reader or just beginning your literary journey, the Book Gallery is a sanctuary for book lovers and a celebration of the written artistry that enriches our lives.

Join us on this literary exploration, where every book is a treasure waiting to be discovered, and every author is an architect of captivating narratives. The Book Gallery awaits your presence—where stories breathe, ideas flourish, and the magic of words unfolds.

book.html

```
<!DOCTYPE html>
```

```
<html lang="en">
```

```
<head>
```

```
  <meta charset="UTF-8">
```

```
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
```

```
  <title>Book Gallery</title>
```

```
  <link rel="stylesheet"
```

```
href="https://cdn.jsdelivr.net/npm/bootstrap@5.2.3/dist/css/bootstrap.min.css">
```

```
  <style>
```

```
    .container a
```

```
    {
```

```
      color: #fff;
```

```
    }
```

```
  </style>
```

```
</head>
```

```
<body>
```

```
<!-- Navigation Bar -->
```

```
<nav class="navbar navbar-expand-lg navbar-light bg-dark ">
```

```
  <div class="container">
```

```
    <a class="navbar-brand" href="nav.html">Book Gallery</a>
```

```
    <button class="navbar-toggler" type="button" data-toggle="collapse"
```

```
data-target="#navbarNav"
```

```
    aria-controls="navbarNav" aria-expanded="false" aria-label="Toggle navigation">
```

```
    <span class="navbar-toggler-icon"></span>
```

```
  </button>
```

```
  <div class="collapse navbar-collapse">
```

```

<ul class="navbar-nav ml-auto">
  <li class="nav-item">
    <a class="nav-link" href="nav.html">Home</a>
  </li>
  <li class="nav-item">
    <a class="nav-link" href="Book_gallery.html">Books</a>
  </li>
  <li class="nav-item">
    <a class="nav-link" href="#">Authors</a>
  </li>
</ul>
</div>
</div>
</nav>
<div class="container">
  <h1 style="padding-top: 20px; text-align: center; padding-bottom: 20px;">Book
  Gallery</h1>
  <div class="row">
    <div class="col-book mb-4" style="width: 20%; padding: 0.5rem;">
      <div class="card h-100 text-center">
        
        <div class="card-body" style="display: flex !important; align-items:
        stretch;">
          <h5 class="card-title">Rich Dad Poor Dad</h5>
          <p class="card-text">Robert T. Kiyosaki</p>
        </div>
      </div>
    </div>
    <div class="col-book mb-4" style="width: 20%; padding: 0.5rem;">
      <div class="card h-100 text-center">
        
        <div class="card-body" style="display: flex !important; align-items:
        stretch;">
          <h5 class="card-title">Atomic Habits</h5>
          <p class="card-text">James Clear</p>
        </div>
      </div>
    </div>
  </div>
</div>

```



```

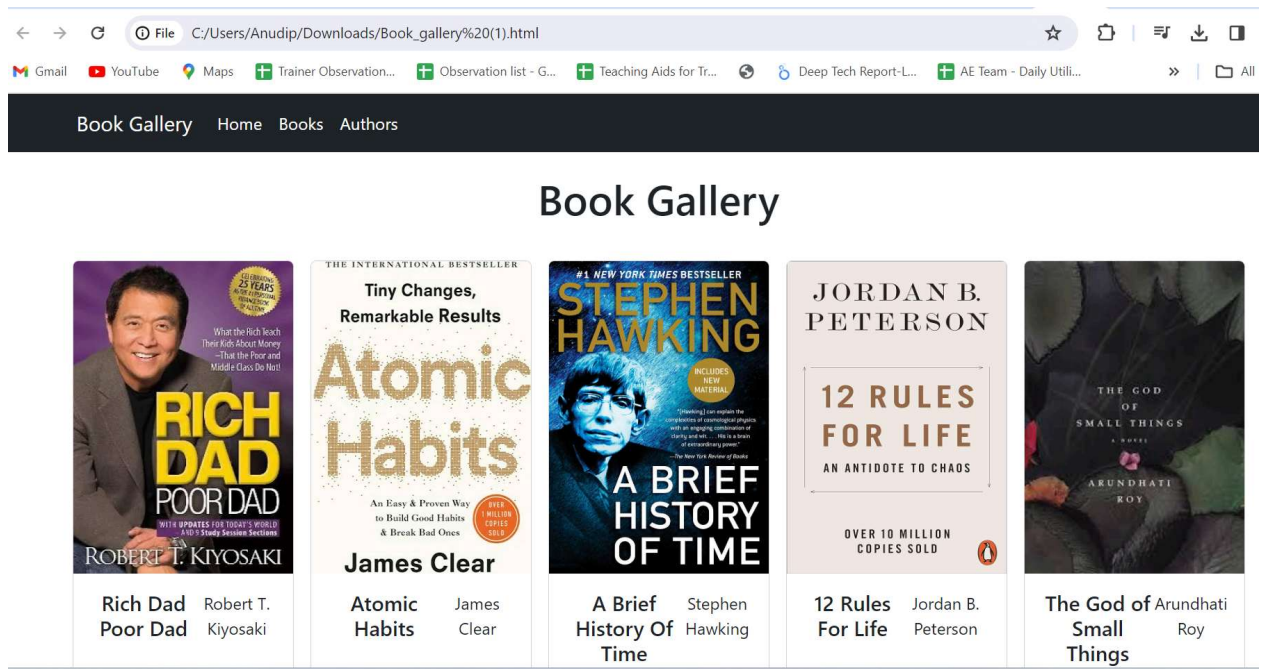
</div>

<div class="col-book mb-4" style="width: 20%; padding: 0.5rem;">
  <div class="card h-100 text-center">
    
    <div class="card-body" style="display: flex !important; align-items:
stretch;">
      <h5 class="card-title">A Brief History Of Time</h5>
      <p class="card-text">Stephen Hawking</p>
    </div>
  </div>
</div>

<div class="col-book mb-4" style="width: 20%; padding: 0.5rem;">
  <div class="card h-100 text-center">
    
    <div class="card-body d-flex" style="display: flex !important; align-items:
stretch;">
      <h5 class="card-title">12 Rules For Life</h5>
      <p class="card-text">Jordan B. Peterson</p>
    </div>
  </div>
</div>

<div class="col-book mb-4" style="width: 20%; padding: 0.5rem;">
  <div class="card h-100 text-center">
    
    <div class="card-body" style="display: flex !important; align-items: stretch;">
      <h5 class="card-title">The God of Small Things</h5>
      <p class="card-text">Arundhati Roy</p>
    </div>
  </div>
</div>
</div>
</body>
</html>

```



V. Event Driven Programming Model in JavaScript

Event-driven programming is a paradigm where the flow of the program is determined by events, such as user actions (mouse clicks, key presses) or system events. In JavaScript, the event-driven model is commonly used for building interactive and responsive web applications.

We have already used events such as onclick events in earlier sessions. Here we will understand the event handling mechanism in detail.

Event Handling

Event handlers in JavaScript are functions that are assigned to handle specific events, such as user actions or system events. These handlers define the behavior that should occur when a particular event takes place. Here's an overview of event handlers in JavaScript:

Inline Event Handlers:

- Inline event handlers are defined directly within the HTML markup using the **on** attribute.
- Although this approach is straightforward, it's generally considered less maintainable

```
<button onclick="handleClick()">Click me</button>
```

Event Listeners:

An Event listener is essentially a JavaScript function that waits for a specific event to occur and then executes a callback function to respond to that event. Event listeners and event handlers are often considered the same thing. However, in essence, they work together to respond to an event. As the name suggests, the listener listens for the event, and the handler is the code that is executed in response to that event.

There are two common built-in event listener methods in JavaScript: `addEventListener` and `removeEventListener`.

The **`addEventListener()`** method enables us to attach an event handler to an element. We can also add multiple event handlers to an element.

`removeEventListener()` allows us to remove an event listener/handler from a specific element.

Let's create a simple alarm clock using event handler and event listener :

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Alarm Clock</title>

  <!-- Font Awesome CSS -->
  <link rel="stylesheet"
href="https://cdnjs.cloudflare.com/ajax/libs/font-awesome/6.2.1/css/all.min.css">

  <style>
    body {
      background-color: #377dff;
      margin: 0;
      padding: 0;
      box-sizing: border-box;
      font-family: "Poppins", sans-serif;
    }

    .hide {
      display: none;
    }
  </style>
```

```
.wrapper {  
  background-color: #ffffff;  
  width: 90%;  
  max-width: 27.25em;  
  padding: 3.12em 5em;  
  position: absolute;  
  left: 50%;  
  top: 50%;  
  transform: translate(-50%, -50%);  
  border-radius: 0.5em;  
}
```

```
.timer-display {  
  font-size: 2.18em;  
  text-align: center;  
  box-shadow: 0 0 20px rgba(0, 139, 253, 0.25);  
  padding: 10px 30px;  
  font-family: "Roboto Mono", monospace;  
}
```

```
.inputs {  
  display: flex;  
  align-items: center;  
  justify-content: center;  
  gap: 1em;  
  margin-top: 2em;  
}
```

```
.inputs input {  
  width: 2.8em;  
  font-size: 1.3em;  
  border: 1px solid #000000;  
  border-radius: 0.1em;  
  padding: 0.4em 0.2em;  
}
```

```
#set {  
  background-color: #377dff;  
  border: none;  
  padding: 1em 2.2em;
```

```
display: block;
margin: 1.5em auto 0 auto;
border-radius: 0.2em;
color: #ffffff;
}

.alarm {
display: grid;
grid-template-columns: 8fr 2fr 2fr;
gap: 1em;
margin-top: 1.5em;
align-items: center;
border-bottom: 1px solid #898f9b;
padding-bottom: 0.6em;
}

.alarm input[type="checkbox"] {
appearance: none;
height: 2em;
width: 3.75em;
background-color: #e2e2ec;
border-radius: 1em;
position: relative;
cursor: pointer;
outline: none;
}

.alarm input[type="checkbox"]:before {
position: absolute;
content: "";
background-color: #757683;
height: 1.43em;
width: 1.43em;
border-radius: 50%;
top: 0.25em;
left: 0.25em;
}

.alarm input[type="checkbox"]:checked {
background-color: #d2e2ff;
```

```

    }

    .alarm input[type="checkbox"]:checked:before {
        background-color: #377dff;
        left: 2em;
    }

    .deleteButton {
        background-color: transparent;
        font-size: 1.5em;
        color: #377dff;
        border: none;
        cursor: pointer;
    }
</style>
</head>
<body>

<div class="wrapper">
    <div class="timer-display" id="timerDisplay">00:00:00 AM</div>
    <div class="container">
        <div class="inputs">
            <input
                type="number"
                id="hourInput"
                placeholder="00"
                min="1"
                max="12"
            />
            <input
                type="number"
                id="minuteInput"
                placeholder="00"
                min="0"
                max="59"
            />
            <select id="ampm">
                <option value="AM">AM</option>
                <option value="PM">PM</option>
            </select>

```

```

    </div>
    <button id="set">Add Alarm</button>
    <div class="activeAlarms"></div>
  </div>
</div>

<script>
  let timerDisplay = document.getElementById("timerDisplay");
  const hourInput = document.getElementById("hourInput");
  const minuteInput = document.getElementById("minuteInput");
  const setAlarm = document.getElementById("set");
  const ampmSelect = document.getElementById("ampm");
  const activeAlarms = document.querySelector(".activeAlarms");
  let alarmsArray = [];
  let alarmSound = new
Audio("https://raw.githubusercontent.com/himalayasingh/music-player-1/master/music/2
.mp3");

  let initialHour = 0,
    initialMinute = 0,
    alarmIndex = 0;

  const appendZero = (value) => (value < 10 ? "0" + value : value);

  const searchObject = (parameter, value) => {
    let alarmObject,
      objIndex,
      exists = false;
    alarmsArray.forEach((alarm, index) => {
      if (alarm[parameter] == value) {
        exists = true;
        alarmObject = alarm;
        objIndex = index;
        return false;
      }
    });
    return [exists, alarmObject, objIndex];
  };

  function displayTimer() {

```

```

let date = new Date();
let hours = date.getHours();
let ampm = hours >= 12 ? "PM" : "AM";
hours = hours % 12 || 12;
let [minutes, seconds] = [
  appendZero(date.getMinutes()),
  appendZero(date.getSeconds()),
];

timerDisplay.innerHTML = `${hours}:${minutes}:${seconds} ${ampm}`;

alarmsArray.forEach((alarm, index) => {
  if (alarm.isActive) {
    if (
      `${alarm.alarmHour}:${alarm.alarmMinute} ${alarm.ampm}` ===
      `${hours}:${minutes} ${ampm}`
    ) {
      alarmSound.play();
      alarmSound.loop = true;
    }
  }
});
}

const inputCheck = (inputValue, max) => {
  inputValue = parseInt(inputValue);
  if (inputValue < 10) {
    inputValue = appendZero(inputValue);
  }
  if (inputValue > max) {
    inputValue = max;
  }
  return inputValue;
};

hourInput.addEventListener("input", () => {
  hourInput.value = inputCheck(hourInput.value, 12);
});

minuteInput.addEventListener("input", () => {

```



```

    minuteInput.value = inputCheck(minuteInput.value, 59);
  });

setAlarm.addEventListener("click", () => {
  alarmIndex += 1;

  let alarmObj = {};
  alarmObj.id = `${alarmIndex}_${hourInput.value}_${minuteInput.value}`;
  alarmObj.alarmHour = hourInput.value;
  alarmObj.alarmMinute = minuteInput.value;
  alarmObj.ampm = ampmSelect.value;
  alarmObj.isActive = false;

  alarmsArray.push(alarmObj);
  createAlarm(alarmObj);
  hourInput.value = appendZero(initialHour);
  minuteInput.value = appendZero(initialMinute);
});

const createAlarm = (alarmObj) => {
  const { id, alarmHour, alarmMinute, ampm } = alarmObj;
  let alarmDiv = document.createElement("div");
  alarmDiv.classList.add("alarm");
  alarmDiv.setAttribute("data-id", id);
  alarmDiv.innerHTML = `<span>${alarmHour}:${alarmMinute} ${ampm}</span>`;

  let checkbox = document.createElement("input");
  checkbox.setAttribute("type", "checkbox");
  checkbox.addEventListener("click", (e) => {
    if (e.target.checked) {
      startAlarm(e);
    } else {
      stopAlarm(e);
    }
  });
  alarmDiv.appendChild(checkbox);

  let deleteButton = document.createElement("button");
  deleteButton.innerHTML = `<i class="fa-solid fa-trash-can"></i>`;
  deleteButton.classList.add("deleteButton");

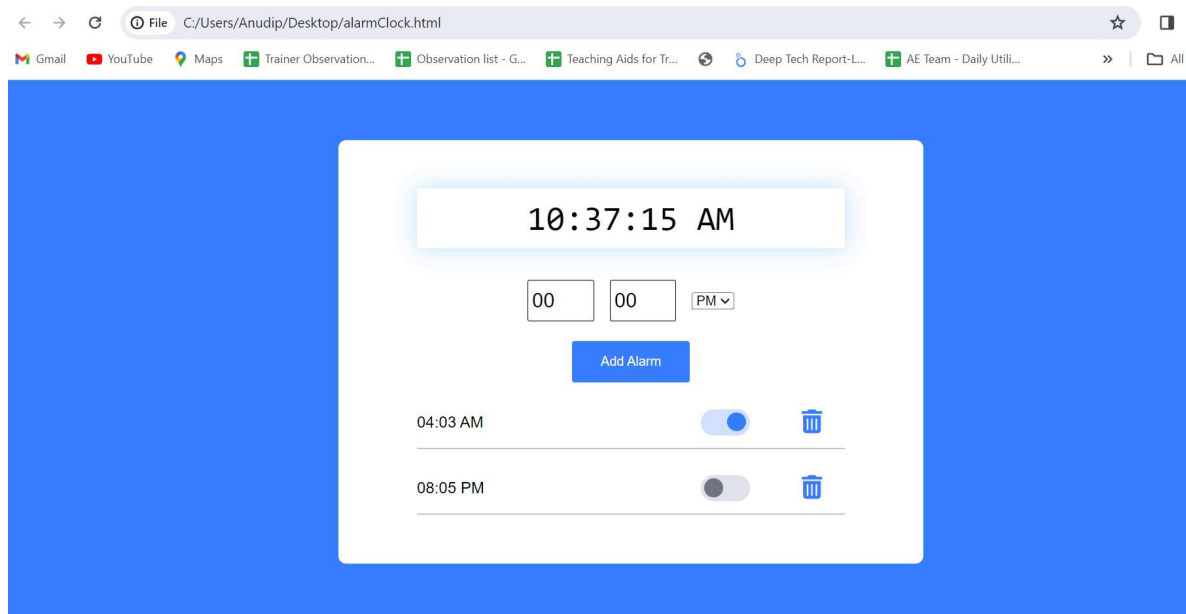
```

```

        deleteButton.addEventListener("click", (e) => deleteAlarm(e));
        alarmDiv.appendChild(deleteButton);
        activeAlarms.appendChild(alarmDiv);
    };
    const startAlarm = (e) => {
        let searchId = e.target.parentElement.getAttribute("data-id");
        let [exists, obj, index] = searchObject("id", searchId);
        if (exists) {
            alarmsArray[index].isActive = true;
        }
    };
    const stopAlarm = (e) => {
        let searchId = e.target.parentElement.getAttribute("data-id");
        let [exists, obj, index] = searchObject("id", searchId);
        if (exists) {
            alarmsArray[index].isActive = false;
            alarmSound.pause();
        }
    };
    const deleteAlarm = (e) => {
        let searchId = e.target.parentElement.parentElement.getAttribute("data-id");
        let [exists, obj, index] = searchObject("id", searchId);
        if (exists) {
            e.target.parentElement.parentElement.remove();
            alarmsArray.splice(index, 1);
        }
    };
    window.onload = () => {
        setInterval(displayTimer, 1000);
        initialHour = 0;
        initialMinute = 0;
        alarmIndex = 0;
        alarmsArray = [];
        hourInput.value = appendZero(initialHour);
        minuteInput.value = appendZero(initialMinute);
        ampmSelect.value = "AM"; // Set default value to AM
    };
</script>
</body>
</html>

```

Output:



In this example:

- The HTML file includes a button element with the ID "Add Alarm."
- The JavaScript code uses `document.getElementById` to obtain a reference to the button.
- An event listener is added using `addEventListener`. It listens for the 'click' event on the button.
- When the button is clicked, the anonymous function inside `addEventListener` is executed, showing an alert.

If you click the button you will see the message.

This simple example demonstrates the event-driven nature of JavaScript. Instead of a linear flow, the program waits for a specific event (button click, in this case) to occur and responds accordingly.

In more complex applications, event-driven programming is used extensively for handling various user interactions, asynchronous operations, and system events, making the application interactive and responsive. We will have a detailed discussion on events and events listeners in this session.

ChatGPT Exercise:

Please visit the site - <https://chat.openai.com/> and create your account if not yet created. Next, start with the first question on the prompt:

I am learning HTML , JavaScript and CSS. Could you provide me with an HTML code with JavaScript that creates a Clock that shows the current time?

I am learning HTML , JavaScript and CSS. Could you provide me with an HTML code with JavaScript that creates a Clock that shows the current time?



ChatGPT can make mistakes. Consider checking important information.

Submit the query by clicking the arrow icon. Now go through the response received from the AI engine. Output will be something like the following screenshot. But please note that there can be some minor programming errors in this free version of ChatGPT. Go through the code and correct the code if required.

After the 2 hr ILT, the student has to do a 1 hour live lab. Please refer to the Session 8 Lab doc in the LMS.